

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

05

객체 지향 프로그래밍



05-7 인터페이스

05-8 다형성

05-9 추상 클래스

05-7 인터페이스

인터페이스는 왜 필요한가?

- 클래스가 추가될 때마다 메서드를 추가하는 어려움을 극복하기 위해
- 어떤 동물의 사육사가 하는 일

```
난 동물원(zoo)의 사육사(zookeeper)이다.
육식 동물(predator)이 들어오면 난 먹이를 던져준다(feed).
호랑이(tiger)가 오면 사과(apple)를 던져준다.
사자(lion)가 오면 바나나(banana)를 던져준다.
```

  - Animal, Tiger, Lion, ZooKeeper 클래스를 작성하여 코드로 표현

05-7 인터페이스

▪ 인터페이스 작성하기

- **interface** 키워드로 작성

Do it! 코딩해 보세요

```
interface Predator {
}

class Animal {
   String name;

   void setName(String name) {
      this.name = name;
   }
}

(... 생략 ...)
```

- 작성한 인터페이스를 구현할 때는 **implements** 키워드를 사용

Do it! 코딩해 보세요

```
(... 생략 ...)

class Tiger extends Animal implements Predator{
}

class Lion extends Animal implements Predator{
}

(... 생략 ...)
```

05-7 인터페이스

▪ 인터페이스 작성하기

- Predator라는 인터페이스에도 IS-A 관계 적용
  - 'Tiger is a Predator', 'Lion is a Predator'가 성립
    - tiger: Tiger 클래스의 객체이자 Predator 인터페이스의 객체
    - lion: Lion 클래스의 객체이자 Predator 인터페이스의 객체

- 육식동물 클래스가 추가될 때마 다음과 같이 Predator 인터페이스 구현

```
class Crocodile extends Animal implements Predator {
}
```

Crocodile 클래스는 실제 코드에 적용하지 말고 눈으로만 살펴보자.

05-7 인터페이스

▪ 인터페이스의 메서드

- feed 메서드가 feed apple 대신 "feed "+predator.getFood()를 출력하도록 코드 수정

Do it! 코딩해 보세요

```
(... 생략 ...)

class Tiger extends Animal implements Predator{
    public String getFood() {
      return "apple";
   }
}

class Lion extends Animal implements Predator{
    public String getFood() {
      return "banana";
   }
}
```

```
(... 생략 ...)

class ZooKeeper {
      public void feed(Predator predator) {
         System.out.println("feed "+predator.getFood());
      }
}

(... 생략 ...)
```

실행 결과

```
feed apple
feed banana
```

05-7 인터페이스

인터페이스 더 파고들기

- ZooKeeper 클래스가 동물 클래스에 의존적인 클래스에서 동물 클래스와 상관없는 독립적인 클래스가 된 것이 인터페이스의 핵심

- 컴퓨터와 자바의 인터페이스 비교

물리적 세계	자바 세계
컴퓨터	ZooKeeper
USB 포트	Predator
하드디스크, 메모리 스틱, 스마트폰, …	Tiger, Lion, Crocodile, …

05-7 인터페이스

디폴트 메서드(default method)

- 자바 8 버전 이후부터 사용할 수 있음
- 인터페이스의 메서드는 구현체를 가질 수 없지만 디폴트 메서드를 사용하면 실제 구현된 형태의 메서드를 가질 수 있음
- 메서드명 가장 앞에 default라고 표기
- 디폴트 메서드는 오버라이딩이 가능

Do it! 코딩해 보세요

```
interface Predator {
    String getFood();

    default void printFood() {
        System.out.printf("my food is %s\n", getFood());
    }
}
(... 생략 ...)
```

05-7 인터페이스

▪ 스태틱 메서드(static method)

- 자바 8 버전 이후부터 사용할 수 있음
- 인터페이스명.스태틱메서드명
- 일반 클래스의 스태틱 메서드를 사용하는 것과 동일하게 사용할 수 있음

Do it! 코딩해 보세요

```
interface Predator {
    String getFood();
    default void printFood() {
        System.out.printf("my food is %s\n", getFood());
    }
    int LEG_COUNT = 4;  // 인터페이스 상수

    static int speed() {
        return LEG_COUNT * 30;
    }
}
(... 생략 ...)
```

05-8 다형성

다형성(polymorphism)

- 하나의 객체가 여러 개의 자료형 타입을 가질 수 있는 것
- 다형성을 이용하면 복잡한 형태의 분기문을 간단하게 처리할 수 있음
- instanceof를 활용했던 Bouncer 클래스의 barkAnimal 메서드와 인터페이스를 사용해 수정한 barkAnimal 메서드 비교

변경 전

```
void barkAnimal(Animal animal) {
    if (animal instanceof Tiger) {
        System.out.println("어흥");
    } else if (animal instanceof Lion) {
        System.out.println("으르렁");
    }
}
```



변경 후

```
void barkAnimal(Barkable animal) {
    animal.bark();
}
```

05-8 다형성

- **다형성(polymorphism)**
  - getFood 메서드와 bark 메서드 모두 사용하고 싶을 경우

Do it! 코딩해 보세요

```
interface Predator {
(... 생략 ...)
}

interface Barkable {
    void bark();
}

interface BarkablePredator extends Predator, Barkable {
}
(... 생략 ...)
```

- getFood, bark 메서드를 모두 포함하는 새로운 인터페이스 BarkablePredator를 만들어 사용

05-8 다형성

▪ 다형성(polymorphism)

▪ Lion 클래스를 BarkablePredator 인터페이스를 구현하도록 수정

Do it! 코딩해 보세요

```
(... 생략 ...)

class Lion extends Animal implements BarkablePredator {
    public String getFood() {
        return "banana";
    }
    public void bark(){
        System.out.println("으르렁");
    }
}

(... 생략 ...)
```

실행 결과

```
어흥
으르렁
```

05-9 추상 클래스

- **추상 클래스(abstract class)**
  - 인터페이스의 역할도 하면서 클래스의 기능도 가지고 있음

Do it! 코딩해 보세요

```
abstract class Predator extends Animal {
    abstract String getFood();

    default void printFood() {
        System.out.printf("my food is %s\n", getFood());
    }

    static int LEG_COUNT = 4;  // 추상 클래스의 상수는 static 선언이 필요하다.
    static int speed() {
        return LEG_COUNT * 30;
    }
}
(... 생략 ...)
```

이해를 돕기 위해 여기에는 그대로 적었지만 실습할 때는 삭제

- 추상 클래스를 만들려면 클래스 앞에 **abstract**를 표기
- 추상 클래스에 abstract로 선언된 메서드는 인터페이스의 메서드와 마찬가지로 반드시 구현해야 함
- 추상 클래스에는 abstract 메서드 외에 실제 메서드도 사용할 수 있음



감사합니다.